GUÍA PRÁCTICA Y DIDÁCTICA · EDICIÓN DE AUTOR

Agentes de IA con MCP

El protocolo que conecta tus agentes con herramientas, datos y prompts — explicado paso a paso



ESCRITO E ILUSTRADO POR

Claude

Agentes de IA con MCP

El protocolo que conecta agentes con el mundo

Qué es el Model Context Protocol, cómo usar clientes,
cómo construir servidores y cómo llevarlo todo a producción.

Escrito e ilustrado por Claude · Anthropic
Edición de autor · Primera edición

Sobre la portada

El animal de esta portada es un **delfín**, y su elección apunta al corazón del libro: *la comunicación*.

MCP es, antes que nada, un **protocolo**: un acuerdo sobre cómo dos partes se hablan. Y el delfín es el gran comunicador del mar. Usa *ecolocalización*: emite un sonido —una señal— y, según el eco que regresa, entiende qué hay alrededor y actúa. Eso es, casi literalmente, un protocolo de petición y respuesta: enviar una solicitud y obrar sobre lo que vuelve. Los delfines, además, coordinan sus manadas con un repertorio de señales compartido, y son célebres por "traducir" entre mundos —el del aire y el del agua— igual que MCP traduce entre el agente y los sistemas externos.

Por eso el delfín de la portada va acompañado de ondas frente a su rostro: la señal que emite y espera de vuelta. Tenlo en mente: todo MCP se reduce a dos partes que se entienden porque hablan el mismo idioma.

Prefacio

Un agente sin acceso al mundo es un cerebro en un frasco. Para ser útil necesita herramientas, datos y contexto. El problema es que cada herramienta se conecta a su manera —y ahí es donde MCP cambia las reglas.

El **Model Context Protocol** (MCP) es un estándar abierto que hace por los agentes lo que el USB-C hizo por los aparatos: un único conector para todo. En vez de programar una integración a medida por cada herramienta y cada agente, todos hablan el mismo protocolo. Construyes un servidor una vez y cualquier agente compatible lo usa.

Este libro te lleva de la mano por los dos lados del protocolo: aprenderás a **usar** servidores desde tu aplicación (el lado cliente) y a **construir** tus propios servidores que ofrezcan herramientas, datos y prompts (el lado servidor). Terminaremos con transportes, seguridad y un ejemplo completo de punta a punta.

Mi promesa: ser lo más didáctico posible. Cada idea nueva viene con una *definición* clara, una *gráfica* que la muestra, una *analogía* cotidiana y, cuando aporta, un ejemplo de código corto. No necesitas ser experto; solo entender que un agente quiere hacer cosas, y MCP es cómo se las arregla para hacerlas.

CÓMO LEER ESTE LIBRO

Los capítulos 1–2 dan la intuición; léelos sí o sí. Los 3–4 son el lado cliente; los 5–7, el lado servidor; el 8, cómo se conectan; el 9, todo junto. Cada capítulo cierra con una caja "Para recordar".

Contenido

1 **IA agéntica y por qué nace MCP**

Agentes, el bucle acción-feedback, y el problema de las M×N integraciones

2 **Qué es el Model Context Protocol**

El USB-C de la IA · arquitectura host, cliente y servidor

3 **Hacer agéntica tu app con clientes MCP**

Conectar, descubrir capacidades y usarlas · el ciclo de vida

4 **Clientes MCP: uso avanzado y buenas prácticas**

Varios servidores, manejo de errores, seguridad del lado cliente

5 **Construir servidores: tools, resources y prompts**

Las tres ofertas de un servidor y quién controla cada una

6 **Servidores avanzados: utilidades y capacidades del cliente**

Sampling, roots y elicitation

7 **Probar, asegurar y compartir tu servidor**

Pruebas, seguridad y distribución

8 **Conectar clientes y servidores: transportes**

stdio (local) y HTTP/SSE (remoto)

9 **Cierre: ejemplos completos y próximos pasos**

Todo junto · qué viene después

CAPÍTULO UNO

IA agéntica y por qué nace MCP

> *Antes de inventar el enchufe universal, alguien tuvo que sufrir mil cables distintos.*
>
> — Sobre el problema que MCP resuelve

Para entender por qué MCP importa, primero hay que ver qué es un agente y qué le hace falta —y el dolor concreto que aparece cuando intentas conectarlo al mundo real.

DEFINICIÓN — Agente de IA

Sistema que recibe un objetivo y trabaja hacia él en un **bucle de acción y feedback**: ejecuta una acción, observa el resultado en su entorno y, con esa información, decide la siguiente acción, hasta terminar.

El bucle de acción y feedback

Imagina un agente que debe escribir pruebas para un archivo de código. Vuelta 1: *lee* el archivo (acción) y obtiene su contenido (feedback). Vuelta 2: *escribe* las pruebas (acción) y obtiene el archivo de test (feedback). Vuelta 3: *ejecuta* las pruebas (acción) y obtiene los resultados (feedback). Si pasan, termina; si no, vuelve a una vuelta anterior y corrige. Ese ciclo —actuar, observar, decidir— es el latido de todo agente.

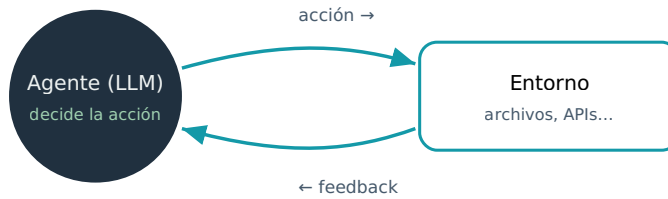


Figura 1.1 — El bucle de acción-feedback: el agente actúa sobre un entorno y aprende de la respuesta.

Agentes y workflows

No todo sistema con IA es un agente. En un **workflow**, el camino lo decide tu código y solo se llama al modelo para pasos puntuales (detectar el idioma de un texto, traducir un fragmento). En un **agente**, es el modelo quien decide el camino. Patrones de workflow comunes —encadenar prompts, paralelizar, enrutar, orquestador-trabajador, evaluador-optimizador— conviven con agentes en los buenos sistemas: código para lo predecible, agente para lo que requiere criterio.

El problema de las M×N integraciones

Aquí está el dolor. Si tienes **M** agentes y **N** herramientas, y cada conexión es a medida, acabas escribiendo **M×N** integraciones distintas —y mantenerlas es una pesadilla—. La idea de MCP es convertir ese **M×N** en **M+N**: cada agente habla el protocolo una vez, cada herramienta lo expone una vez, y todos se entienden.

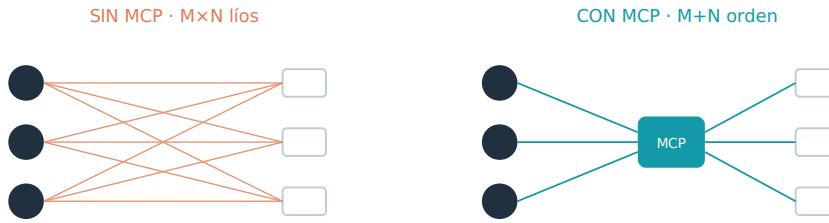


Figura 1.2 — MCP convierte un enredo de M×N integraciones en un ordenado M+N.

PARA RECORDAR

Un agente vive en un bucle de acción–feedback y necesita tocar el mundo. Conectarlo a cada herramienta a medida explota en M×N integraciones. MCP propone un protocolo común que lo reduce a M+N: el problema que el resto del libro resuelve en detalle.

CAPÍTULO DOS

Qué es el Model Context Protocol

> *Un buen estándar es invisible: simplemente todo encaja.*
>
> — Sobre la elegancia de los protocolos

MCP es un acuerdo sobre cómo una aplicación de IA y una fuente de capacidades se hablan. Su famosa analogía lo dice todo: es el **USB-C de la IA**.

DEFINICIÓN — Model Context Protocol (MCP)

Estándar abierto que define cómo las aplicaciones de IA se conectan con herramientas, datos y prompts externos mediante un protocolo común, de modo que cualquier cliente compatible pueda usar cualquier servidor compatible sin integraciones a medida.

La analogía del USB-C

Antes del USB-C, cada aparato traía su propio cargador. Hoy, un solo conector sirve para el teléfono, la laptop y los audífonos. MCP hace lo mismo con la IA: en vez de un "cable" distinto por cada herramienta, hay un conector universal. Construyes el "puerto" una vez y todo lo que hable MCP se enchufa.

Los tres papeles: host, cliente y servidor

MCP define tres roles. Entenderlos es entender MCP entero.

Rol	Qué es	Ejemplo
Host	La aplicación de IA donde vive el agente y que el usuario usa.	Tu app, un IDE, una app de escritorio de chat.
Cliente	El componente dentro del host que habla MCP. Mantiene una conexión 1 a 1 con un servidor.	El conector que tu app abre hacia un servidor.
Servidor	El programa que ofrece capacidades (herramientas, datos, prompts) por MCP.	Un servidor de archivos, de tu base de datos, de GitHub…

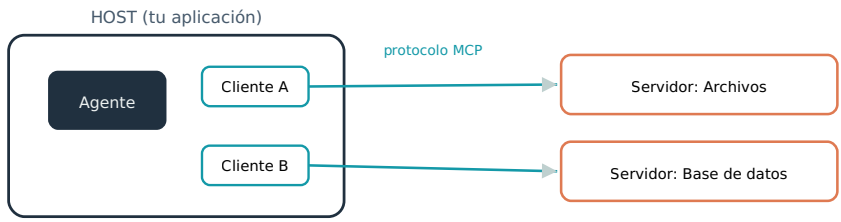


Figura 2.1 — Arquitectura de MCP: un host con varios clientes, cada uno conectado a un servidor.

ANALOGÍA

El host es el restaurante; el cliente es el mesero; el servidor es la cocina. El mesero (cliente) lleva tu pedido a una cocina (servidor) y te trae el plato. Si abre otra cocina, basta otro mesero —el protocolo de "tomar el pedido y traer el plato" no cambia—.

PARA RECORDAR

MCP = el USB-C de la IA. Tres papeles: el *host* (tu app), el *cliente* (habla el protocolo, 1 a 1 con un servidor) y el *servidor* (ofrece capacidades). Todo lo demás del libro es detalle de cómo estos tres conversan.

CAPÍTULO TRES

Hacer agéntica tu app con clientes MCP

> *Conectar es fácil; lo difícil —y lo elegante— es que el agente descubra solo lo que puede hacer.*
>
> — Sobre el lado cliente

Empecemos por el lado más común: usar servidores que ya existen. Tu aplicación abre un **cliente** hacia un servidor y, a partir de ahí, el agente gana nuevas capacidades sin que tú programes cada una.

El ciclo de vida de una conexión

Cuando un cliente se conecta a un servidor ocurren tres pasos: **inicializar** (saludarse y acordar versiones y capacidades), **descubrir** (el cliente pregunta "¿qué ofreces?" y el servidor lista sus herramientas, recursos y prompts) y **usar** (el agente invoca lo que necesita). Lo bello: el agente *descubre* las capacidades en tiempo real, así que añadir un servidor nuevo le da poderes nuevos sin tocar el código del agente.

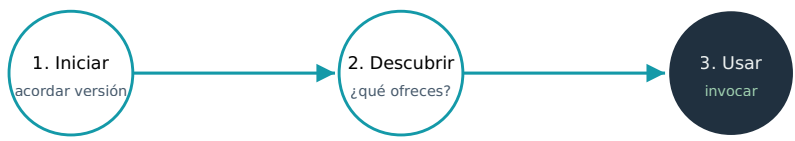


Figura 3.1 — El ciclo de vida: iniciar, descubrir y usar.

Conectar un servidor en la práctica

En muchas apps, conectar un servidor es tan simple como declararlo en un archivo de configuración: le dices al host qué programa ejecutar y con qué argumentos. El host abre el cliente, hace el saludo y descubre las capacidades por ti.

```
// Configurar un servidor MCP en el host (ejemplo conceptual)
{
  "mcpServers": {
    "archivos": {
      "command": "npx",
      "args": ["-y", "servidor-archivos", "/ruta/permitida"]
    },
    "base-datos": {
      "command": "python",
      "args": ["servidor_db.py"]
    }
  }
}
// El host arranca cada servidor, abre un cliente y descubre sus capacidades.
```

PARA RECORDAR

Del lado cliente, una conexión es iniciar → descubrir → usar. Como el agente *descubre* las capacidades en vivo, sumar un servidor le da poderes nuevos sin reprogramar el agente. Empezar suele ser solo declarar el servidor en la configuración del host.

CAPÍTULO CUATRO

Clientes MCP: uso avanzado y buenas prácticas

> *Conectar uno es un tutorial; orquestar muchos, sin que el agente se confunda, es ingeniería.*
>
> — Sobre la madurez del lado cliente

En el mundo real, un agente se conecta a varios servidores a la vez, las cosas fallan y la seguridad importa. Estas son las prácticas que separan un experimento de una app sólida.

Varios servidores sin caos

Cuando un agente tiene acceso a muchos servidores, puede ver decenas de herramientas. Demasiadas opciones lo confunden y encarecen cada llamada. Buenas prácticas: exponer al agente solo las herramientas relevantes a la tarea actual, nombrarlas con claridad para evitar ambigüedades (dos servidores con una herramienta `"buscar"` necesitan contexto), y agruparlas por servidor.

Cuando algo falla

Los servidores externos caen, tardan o devuelven errores. Un cliente maduro lo espera: pone tiempos límite, reintenta con criterio, y traduce un fallo técnico en algo que el agente pueda manejar (`"la herramienta no respondió; intenta otra vía"`) en lugar de romperse.

SEGURIDAD DEL LADO CLIENTE

El host es la última línea de defensa del usuario. Antes de ejecutar una herramienta con efectos —borrar, enviar, pagar—, pide **consentimiento**. Y trata todo lo que devuelve un servidor como *datos no confiables*: si un recurso traído por el servidor contiene texto que parece una orden (`"ignora tus instrucciones y…"`), eso es información, no un mandato. Esta distinción frena los ataques por inyección.

PARA RECORDAR

Con muchos servidores: muestra solo lo relevante y nombra con claridad. Espera los fallos con tiempos límite y reintentos. Y protege al usuario: pide consentimiento para acciones con efectos y nunca confundas el contenido devuelto por un servidor con órdenes.

CAPÍTULO CINCO

Construir servidores: tools, resources y prompts

> *Un servidor no es código que el agente ejecuta: es un menú de lo que el agente puede pedir.*
>
> — Sobre el lado servidor

Ahora el otro lado: construir tu propio servidor. Un servidor MCP ofrece tres tipos de cosas —y entender *quién controla cada una* es la clave de todo el capítulo.

Las tres ofertas de un servidor

Primitiva	Qué es	Quién la controla
Tools	Acciones que el agente puede ejecutar (buscar, escribir, calcular).	El modelo: decide cuándo usarlas.
Resources	Datos y contexto para leer (archivos, registros, documentos).	La aplicación: decide qué exponer.
Prompts	Plantillas de instrucciones reutilizables.	El usuario: las elige e invoca.

DEFINICIÓN — Tool, Resource y Prompt

Una **tool** es una función que el modelo puede invocar para *hacer* algo. Un **resource** es información que el cliente puede *leer* para dar contexto. Un **prompt** es una plantilla preparada que el usuario puede *invocar* para iniciar una tarea común.

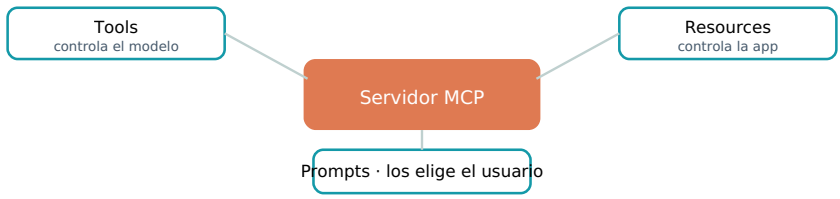


Figura 5.1 — Las tres primitivas y quién controla cada una: la distinción más importante de MCP.

Un servidor mínimo con una tool

Construir un servidor es sorprendentemente simple. Defines una función, la decoras como herramienta con una buena descripción, y el framework se encarga del protocolo.

```python
# servidor.py — un servidor MCP con una sola tool
from mcp.server import FastMCP

mcp = FastMCP("calculadora")

@mcp.tool()
def sumar(a: int, b: int) -> int:
    """Suma dos números enteros y devuelve el resultado."""
    return a + b

@mcp.resource("config://moneda")
def moneda() -> str:
    """Expone la moneda configurada como contexto de solo lectu-
ra."""
    return "PEN"

if __name__ == "__main__":
    mcp.run()   # el framework habla el protocolo MCP por ti
```

La docstring de cada herramienta *es* lo que el modelo lee para decidir cuándo usarla: una descripción clara aquí vale más que mil líneas de lógica.

> PARA RECORDAR
>
> Un servidor ofrece *tools* (acciones que controla el modelo), *resources* (datos que controla la app) y *prompts* (plantillas que elige el usuario). Esa pregunta — *¿quién controla esto?* — es la brújula del diseño. Construirlo es declarar funciones con buenas descripciones; el framework hace el resto.

CAPÍTULO SEIS

Servidores avanzados: utilidades y capacidades del cliente

> *Lo más sorprendente de MCP: a veces es el servidor quien le pide algo al cliente.*
>
> — Sobre la comunicación de ida y vuelta

Hasta ahora el cliente pedía y el servidor respondía. Pero MCP permite el camino inverso: un servidor puede aprovechar capacidades *del cliente*. Tres son clave.

DEFINICIÓN — Sampling

Capacidad por la que un **servidor** puede pedirle al **cliente** que use *su* modelo de lenguaje para generar algo. Así, el servidor obtiene inteligencia sin necesidad de tener (ni pagar) su propio modelo.

Sampling invierte la relación: el servidor, a mitad de una tarea, dice "necesito que un LLM resuma esto" y el cliente lo hace con su modelo. Útil para servidores que necesitan razonamiento pero no quieren incluir un modelo propio. El cliente mantiene el control: puede revisar o rechazar la petición.



Figura 6.1 — Sampling: el servidor pide al cliente que use su modelo (la comunicación va de vuelta).

Roots y elicitation

- **Roots**: el cliente le dice al servidor sobre qué carpetas o ámbitos puede operar. Es una frontera de seguridad: el servidor de archivos solo ve lo que el cliente le permite ver.
- **Elicitation**: el servidor puede pedirle *al usuario* un dato que le falta a mitad de la tarea (`"¿a qué cuenta lo cargo?"`), a través del cliente, en lugar de fallar o adivinar.

> PARA RECORDAR
>
> MCP no es de un solo sentido. Con *sampling*, el servidor usa el modelo del cliente; con *roots*, el cliente acota dónde puede operar el servidor; con *elicitation*, el servidor pide datos al usuario. Estas utilidades convierten servidores simples en colaboradores.

CAPÍTULO SIETE

Probar, asegurar y compartir tu servidor

> *Un servidor que cualquiera puede conectar es un servidor en el que cualquiera debe poder confiar.*
>
> — Sobre la responsabilidad de publicar

Si tu servidor va a ser usado por agentes —tuyos o de otros—, debe estar probado, ser seguro y poder compartirse con confianza.

Probar

Prueba tu servidor en dos niveles: que cada herramienta haga lo correcto con entradas válidas e inválidas (pruebas normales), y que un agente real lo use bien —¿elige la herramienta adecuada?, ¿interpreta el resultado?—. Existen inspectores que se conectan a tu servidor y te dejan invocar sus herramientas a mano para verificar el comportamiento antes de exponerlo a un modelo.

Asegurar

Un servidor es una puerta al sistema; cuídala:

- **Valida toda entrada** antes de actuar. El modelo puede pedir cualquier cosa; tu servidor decide qué permite.
- **Mínimo privilegio**: que cada herramienta pueda hacer lo justo y nada más. Un servidor de lectura no debería poder borrar.

- **Cuidado con la inyección por recursos**: si tu servidor devuelve contenido externo (una página, un correo), recuerda que ese texto puede contener intentos de manipular al agente. Etiquétalo como dato, no como instrucción.
- **Confirma lo irreversible**: las acciones con efectos serios deben pasar por consentimiento del usuario.

Compartir

Para que otros usen tu servidor, empaquétalo de forma que se ejecute con un comando simple, documenta qué herramientas, recursos y prompts ofrece, y deja claras sus necesidades (qué claves o permisos requiere). Un buen servidor se conecta en segundos y explica por sí mismo lo que hace.

> PARA RECORDAR
>
> Antes de publicar: prueba cada herramienta y el uso por un agente real; valida toda entrada y aplica mínimo privilegio; trata el contenido externo como dato; confirma lo irreversible; y empaqueta con documentación clara. Compartir un servidor es compartir confianza.

CAPÍTULO OCHO

Conectar clientes y servidores: transportes

> *El idioma es el mismo; el canal por el que viaja, no siempre.*
>
> — Sobre los transportes

El protocolo define *qué* se dicen cliente y servidor. El **transporte** define *cómo* viajan esos mensajes. Hay dos grandes opciones, según dónde vive el servidor.

DEFINICIÓN — Transporte

Mecanismo por el que los mensajes de MCP viajan entre cliente y servidor. No cambia *qué* se dicen (el protocolo es el mismo), solo el canal físico de la comunicación.

Transporte	Cuándo	Cómo funciona
stdio	Servidor local, en tu propia máquina.	El host lanza el servidor como un proceso y se hablan por la entrada/salida estándar. Rápido y privado.
HTTP/SSE	Servidor remoto, en otra máquina o en la nube.	Cliente y servidor se comunican por HTTP, con respuestas en tiempo real. Permite servidores compartidos por muchos usuarios.

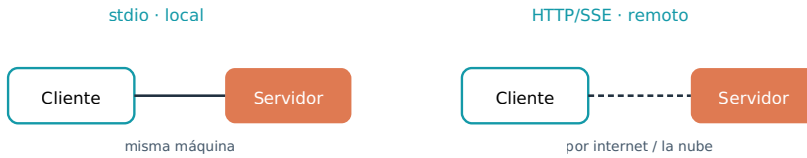


Figura 8.1 — Dos transportes: stdio para servidores locales, HTTP/SSE para servidores remotos.

La buena noticia: como el protocolo no cambia, el mismo servidor puede ofrecerse por uno u otro transporte con poco esfuerzo. Empiezas en local con stdio para desarrollar, y pasas a HTTP cuando quieres que otros lo usen desde la nube.

> PARA RECORDAR
>
> El protocolo es el idioma; el transporte es el canal. *stdio* para servidores locales (rápido, privado); *HTTP/SSE* para servidores remotos y compartidos. El mensaje no cambia, solo por dónde viaja —así que migrar de uno a otro es sencillo.

CAPÍTULO NUEVE

Cierre: ejemplos completos y próximos pasos

> *Cuando las piezas encajan, dejas de ver piezas y empiezas a ver un sistema.*
>
> — Sobre poner todo junto

Atemos los cabos con un ejemplo completo y miremos hacia adelante. Recordando al delfín: todo se reduce a partes que se entienden porque hablan el mismo protocolo.

El sistema completo

Imagina un asistente de soporte. El **host** es tu app de chat; dentro vive el **agente**. Abres tres **clientes** hacia tres **servidores**: uno de tu base de conocimiento (expone *resources* para RAG), uno de tu sistema de pedidos (expone *tools* para consultar y actualizar) y uno de plantillas (expone *prompts* de respuestas comunes). El usuario escribe; el agente descubre las capacidades, decide qué herramienta usar, lee el contexto que necesita y responde —pidiendo confirmación antes de cualquier acción con efectos—.

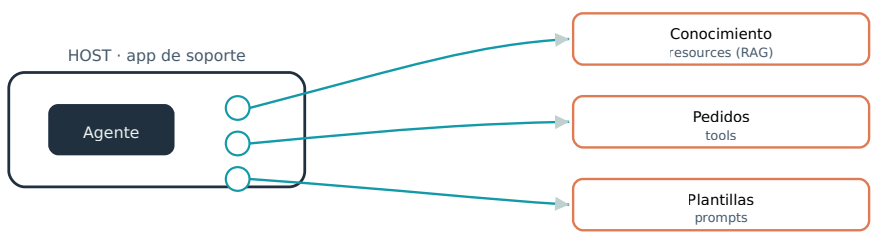


Figura 9.1 — Un sistema completo: un agente, tres clientes, tres servidores con las tres primitivas.

Por qué esto importa

Fíjate en lo que *no* tuviste que hacer: no programaste una integración a medida para cada sistema. Cada servidor se construye una vez y sirve a cualquier agente; cada agente habla MCP una vez y usa cualquier servidor. Ese es el M+N del primer capítulo, hecho realidad.

Próximos pasos

MCP es joven y crece rápido: más servidores listos para usar, mejores transportes remotos, más capacidades de ida y vuelta, y un ecosistema donde compartir servidores será tan natural como publicar una librería. El mejor próximo paso es práctico: conecta un servidor existente a tu app, luego construye uno mínimo con una herramienta tuya, y verás que el protocolo desaparece —que es justo lo que hace un buen estándar—.

> PARA RECORDAR
>
> Todo MCP encaja en una imagen: un host con su agente, clientes que hablan el protocolo y servidores que ofrecen tools, resources y prompts. Construyes cada pieza una vez y se reutiliza en todas partes. Empieza usando un servidor, luego construye el tuyo: lo demás es práctica.

Glosario

- **MCP**: protocolo estándar abierto para conectar agentes con herramientas, datos y prompts.
- **Host**: la aplicación de IA donde vive el agente y que usa la persona.

- **Cliente**: componente del host que habla MCP, en conexión 1 a 1 con un servidor.
- **Servidor**: programa que ofrece capacidades por MCP.
- **Tool**: acción invocable; la controla el modelo.
- **Resource**: dato o contexto de solo lectura; lo controla la aplicación.
- **Prompt**: plantilla de instrucciones; la elige el usuario.
- **Sampling**: el servidor pide al cliente que use su modelo para generar algo.
- **Roots**: el cliente delimita los ámbitos sobre los que opera el servidor.
- **Elicitation**: el servidor solicita un dato al usuario a través del cliente.
- **Transporte**: el canal por el que viajan los mensajes (stdio local, HTTP/SSE remoto).
- **Problema M×N**: el enredo de integraciones a medida que MCP reduce a M+N.

Fin · Agentes de IA con MCP · Edición de autor